NAME: ADOYI NATHANIEL JOHNSON

STUDENT ID: FWSD2511203

STUDENT EMAIL: fwsd2511203@students.icdfa.edu.ng

COURSE CODE: SBT-203-BASIC NETWORKING SKILLS

LAB NAME: LAB2 HTTP ANALYSIS USING WIRESHARK EMBEDDED

DATE: 10/09/26

INSTRUCTOR: AMINU IDRIS

International Cybersecurity and Digital Forensics Academy (ICDFA)

SBT-DF203: Basic Networking Skills for Digital Forensics

LAB 2: HTTP Analysis Using Wireshark - Embedded Image Traffic

Three-Week Delivery Block 1/3 | 5-11 September 2026

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction
Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.

Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 2
Lab Title	HTTP Analysis Using Wireshark - Embedded Image Traffic
Delivery Block	1/3 of 3
Scheduled Dates	5-11 September 2026
Estimated Duration	3-4 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	image_traffic.pcapng generated locally; webpage and image objects

1. Source Materials and Slide Alignment

- HTTP, Wireshark and Digital Forensics Part 2: webpages containing embedded images, multiple HTTP GET requests, TCP segmentation/reassembly and object extraction.
- The slide sequence demonstrates image.html and building.jpg, HTTP 200 responses, large payloads split across TCP segments, Export Objects and size relationships.
- The assignment requires browser-generated capture, insertion of a photograph, extraction of that photograph and explanation of curl/browser differences.

Reading Requirement Before Lab Execution
The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded SBT-DF203 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

A web page and an embedded image were transferred over plaintext HTTP. Your task is to capture the transaction, prove that the browser requested two objects, reconstruct the segmented image response, export the image from the capture and verify that the recovered object matches the original.

3. Learning Outcomes

- Explain why one webpage can generate multiple HTTP requests and responses.
- Capture a browser request for HTML and an embedded image.
- Identify TCP segments that form one reassembled HTTP response.
- Calculate and explain IP length, TCP payload length and HTTP object size relationships.
- Extract an image using Wireshark Export Objects and tshark.
- Hash the original and extracted image and assess integrity.
- Document limitations caused by cache, HTTPS or incomplete capture.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.

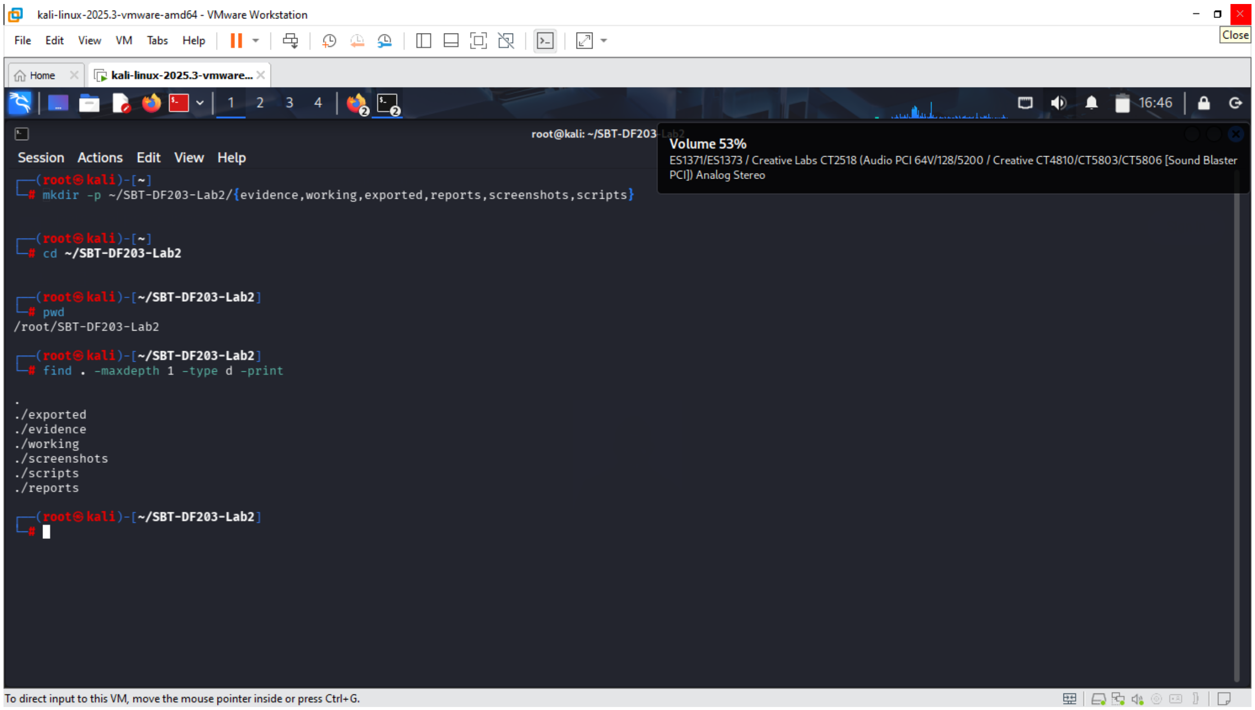
5. Required Tools and Environment

Item	Recommended Tool	Purpose
Operating system	Kali Linux plus optional Windows 10/11 VM	Host the page and generate a browser request.
Web server	Apache2	Serve image.html and the selected image.
Capture tools	Wireshark and tshark	Capture and reconstruct HTTP objects.
Browser	Firefox, Chrome or Edge	Generate separate HTML and image requests.
Utilities	curl, file, identify, sha256sum	Inspect and verify objects.
Image	Student-owned non-sensitive JPG/PNG	Embedded evidence object.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab2
pwd
find . -maxdepth 1 -type d -print
```

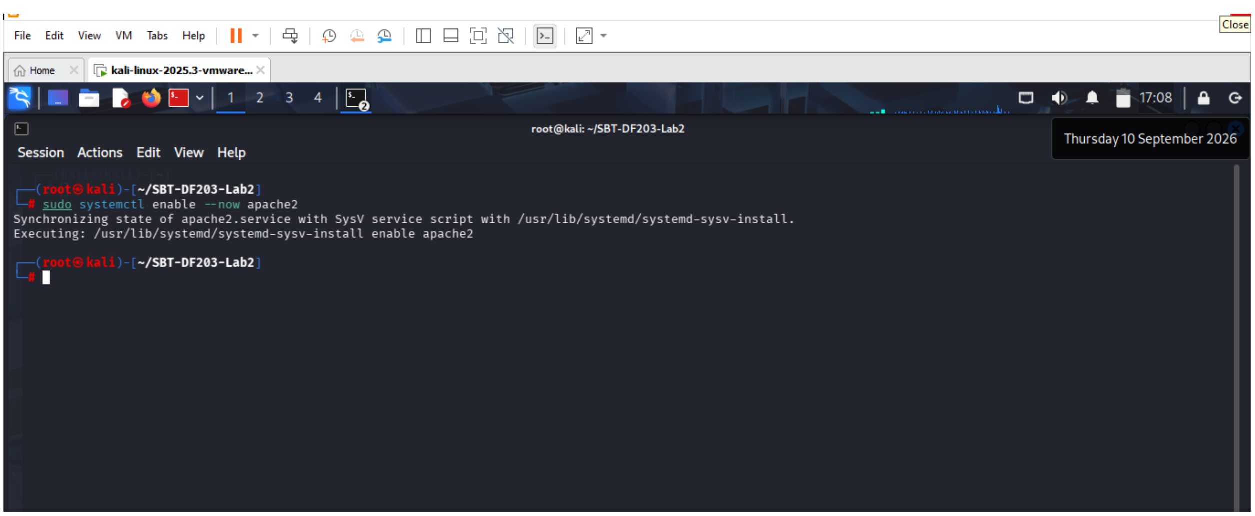
ANSWER:

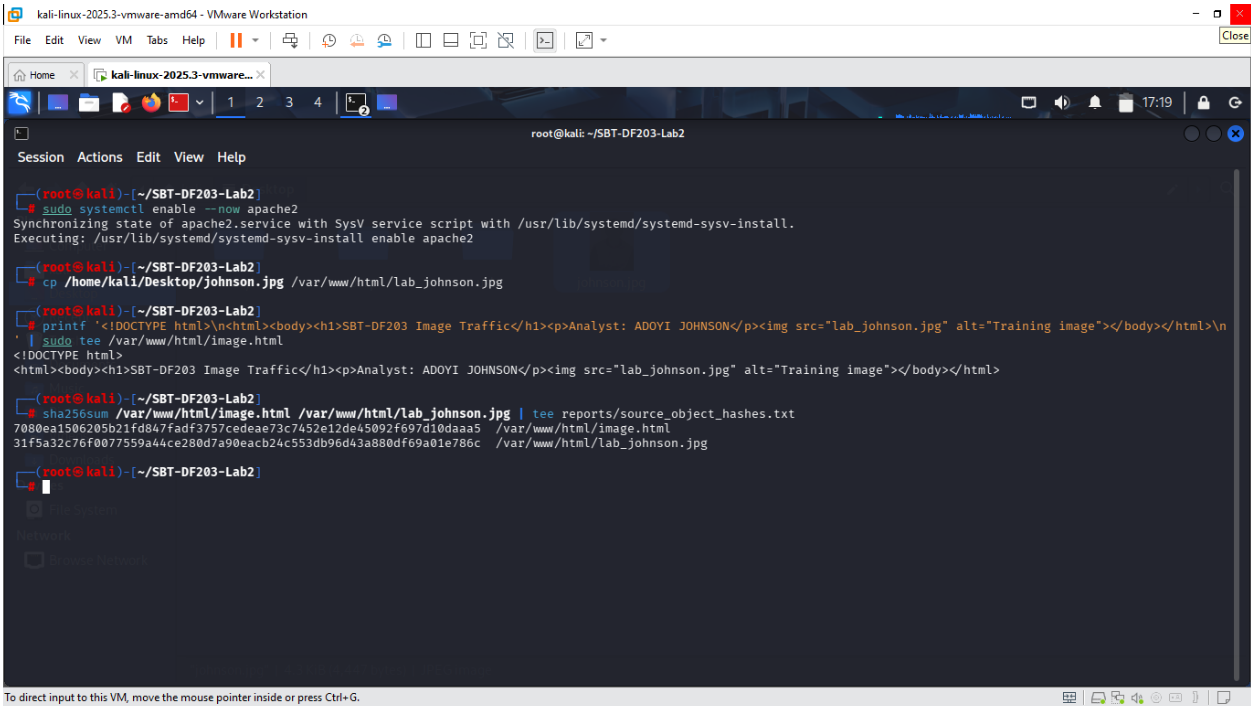


Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y apache2 curl wireshark tshark imagemagick
sudo systemctl enable --now apache2
cp /PATH/TO/YOUR/PHOTO.jpg /var/www/html/lab_photo.jpg
printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: YOUR NAME</p><img
src="lab_photo.jpg" alt="Training image"></body></html>\n' | sudo tee /var/www/html/image.html
sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee reports/source_object_hashes.txt
```

ANSWER:





Screenshot required: The source webpage, source image file information and their SHA-256 hashes.

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab2-YourFullName
Trainee name	ADOYI NATHANIEL JOHNSON
Date and time started	08/09/26
Evidence file name(s)	SBT-DF203-Lab2 FWSD2511203
Source or generation method	KALI LINUXCOMMAND
Original SHA-256	7080ea1506205b21fd847fadf3757cedeae73c7452e12de45092f697d10daaa5 /var/www/html/image.html
Working-copy SHA-256	31f5a32c76f0077559a44ce280d7a90eacb24c553db96d43a880df69a01e786c /var/www/html/lab_johnson.jpg
Analysis workstation/VM	ROOT@KALI
Notes on any changes	NONE

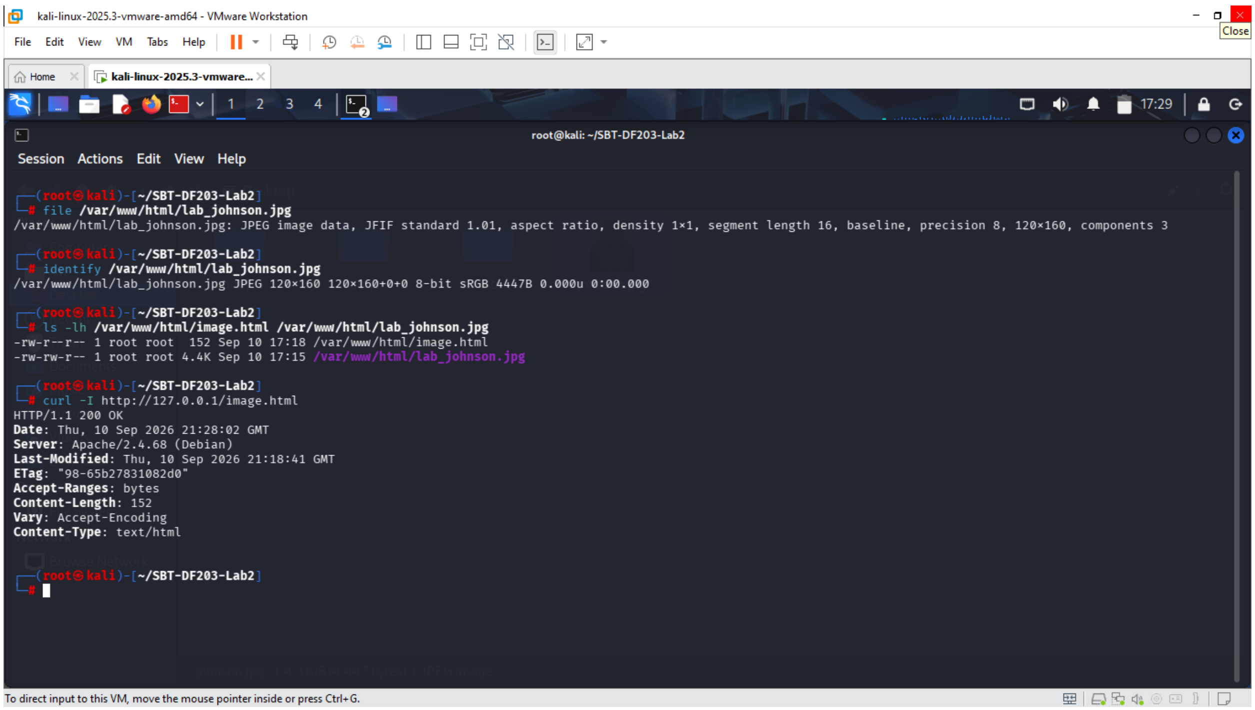
8. Part A - Verify the Webpage and Eliminate Cache Effects

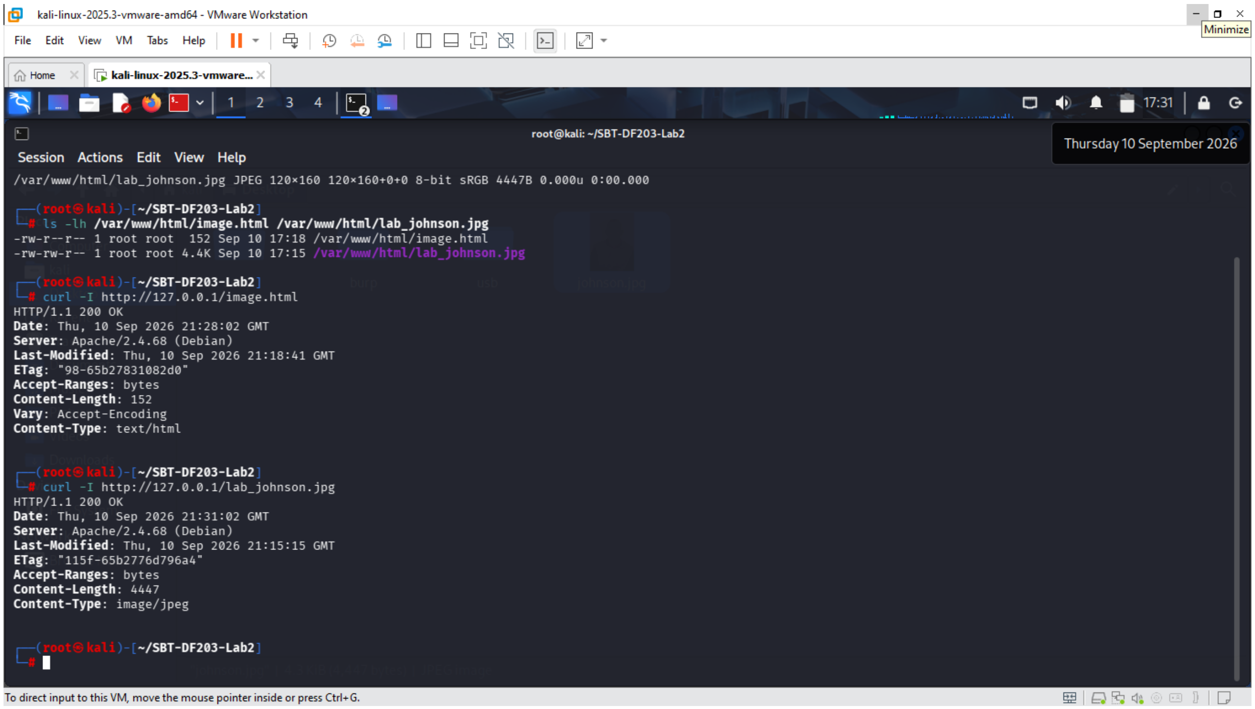
1. Use a non-sensitive image you are authorized to submit.

2. Open a private/incognito browser window or clear the cache before capture.
3. Confirm that image.html displays both text and the embedded image.
4. Record file type, dimensions and size of the source image.

```
file /var/www/html/lab_photo.jpg
identify /var/www/html/lab_photo.jpg
ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
curl -I http://127.0.0.1/image.html
curl -I http://127.0.0.1/lab_photo.jpg
```

ANSWER:





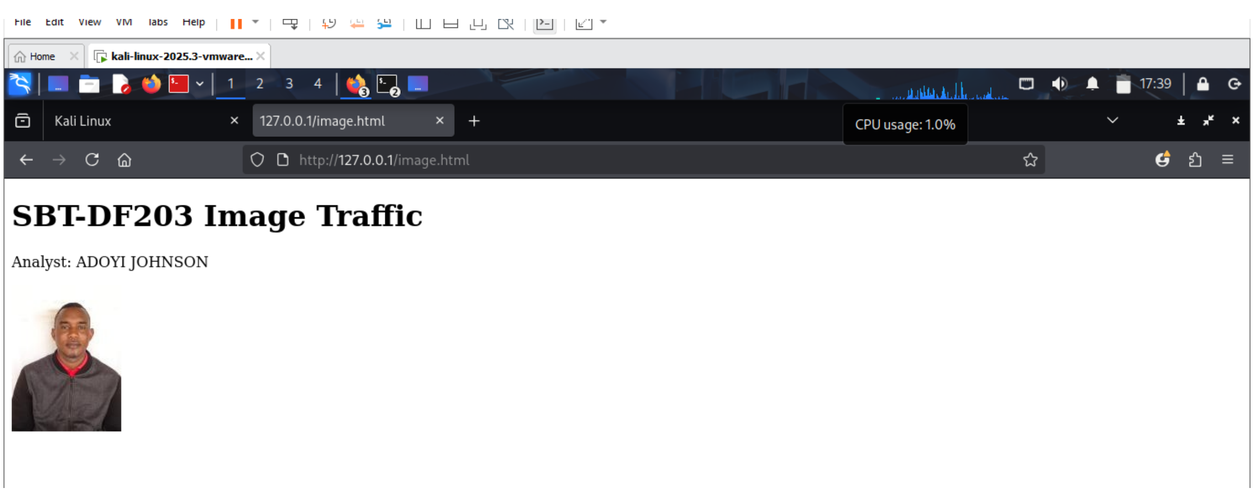
9. Part B - Capture Browser-Generated HTTP Traffic

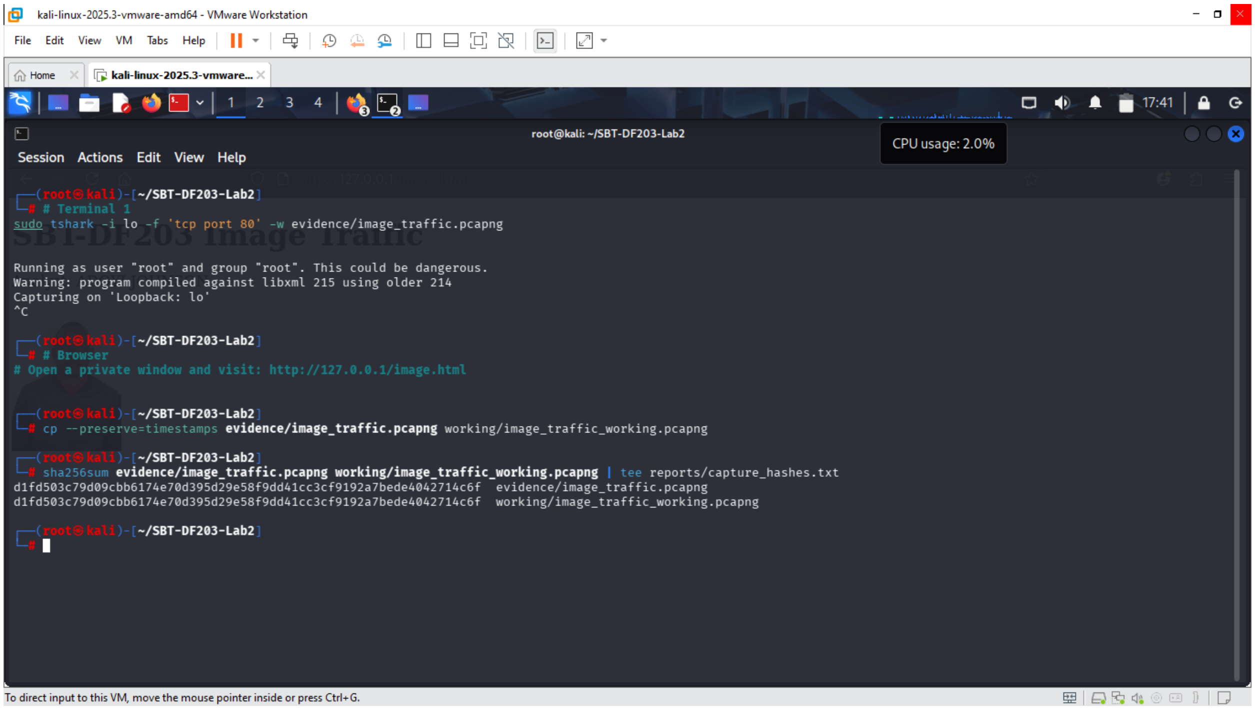
```
# Terminal 1
sudo tshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng

# Browser
# Open a private window and visit: http://127.0.0.1/image.html
# Wait for the image to load, then stop the capture with Ctrl+C.

cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng
sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng | tee
reports/capture_hashes.txt
```

ANSWER:





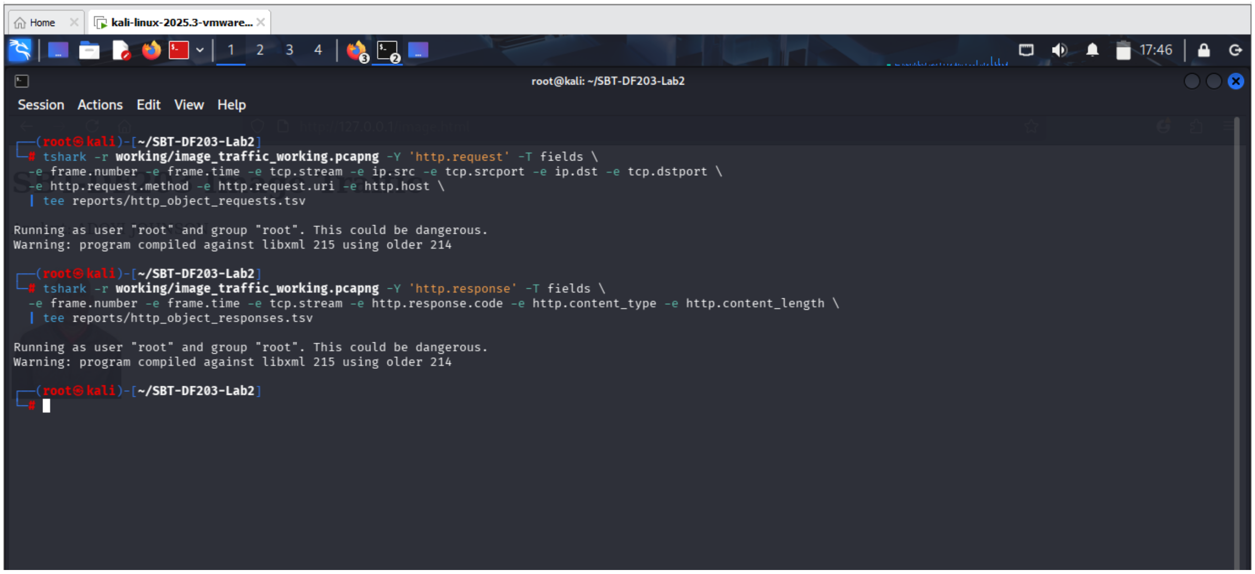
10. Part C - Prove That Two Objects Were Requested

Apply http.request. You should normally see one GET request for image.html and another for lab_photo.jpg. A favicon or browser-generated request may also appear; document it rather than deleting it.

```
tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e http.request.method -e http.request.uri -e http.host \
  | tee reports/http_object_requests.tsv

tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e http.response.code -e http.content_type -e
http.content_length \
  | tee reports/http_object_responses.tsv
```

ANSWER:



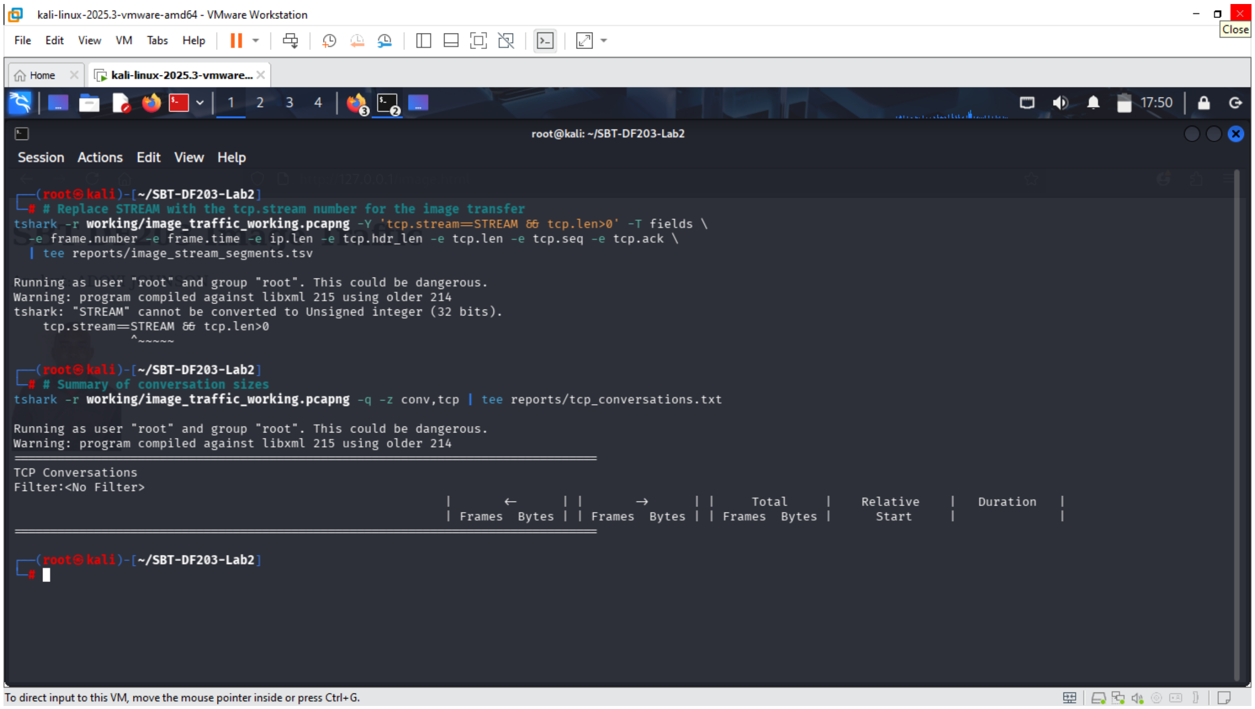
11. Part D - Analyze TCP Segmentation and Reassembly

5. Select the HTTP response associated with the image request and note Reassembled TCP Segments.
6. Record the number and tcp.len value of each contributing segment.
7. Add the TCP payload lengths and compare the result with the HTTP header plus object data.
8. Explain why the exact segment sizes may differ from those in the slide: interface offloading, operating system buffers, MTU and capture location can change segmentation.

```
# Replace STREAM with the tcp.stream number for the image transfer
tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==STREAM && tcp.len>0' -T fields \
  -e frame.number -e frame.time -e ip.len -e tcp.hdr_len -e tcp.len -e tcp.seq -e tcp.ack \
  | tee reports/image_stream_segments.tsv

# Summary of conversation sizes
tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt
```

ANSWER:



12. Part E - Export the Embedded Image

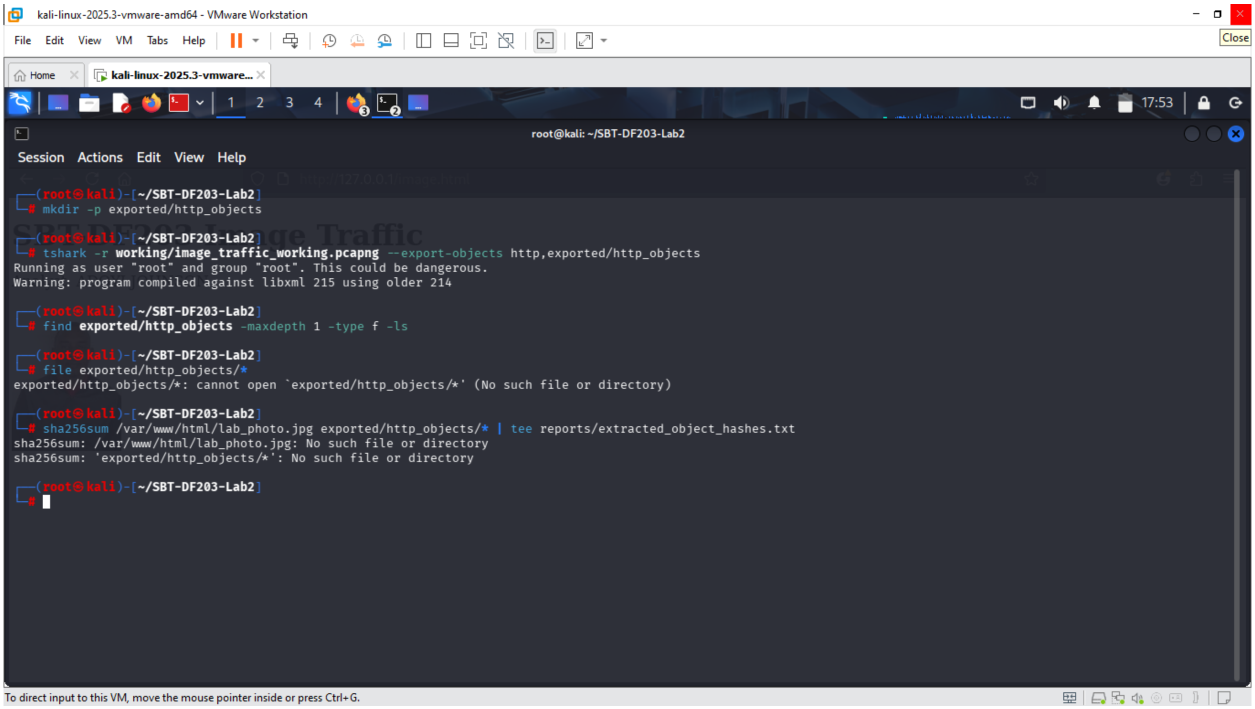
9. In Wireshark select File > Export Objects > HTTP.
10. Locate lab_photo.jpg by hostname, content type or filename and save it under exported.
11. Alternatively, use tshark object export as shown below.
12. Compare file type, size and SHA-256 hash of the exported image with the source image.

```
mkdir -p exported/http_objects
tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
find exported/http_objects -maxdepth 1 -type f -ls
file exported/http_objects/*
sha256sum /var/www/html/lab_photo.jpg exported/http_objects/* | tee reports/extracted_object_hashes.txt
```

Integrity Interpretation

Matching hashes prove byte-for-byte recovery. A different filename is not a problem if size, type and hash match. A different hash requires investigation of incomplete capture, browser content transformation, a different requested object or export error.

ANSWER:



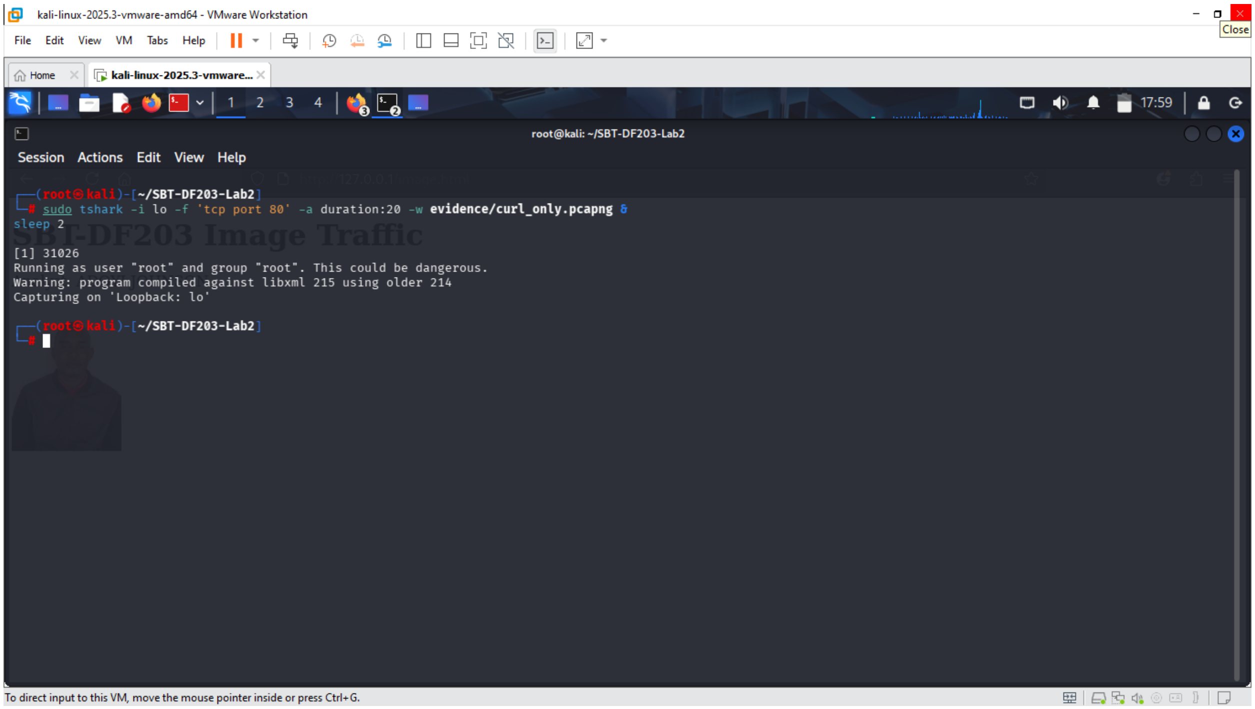
13. Part F - Compare curl and Browser Behaviour

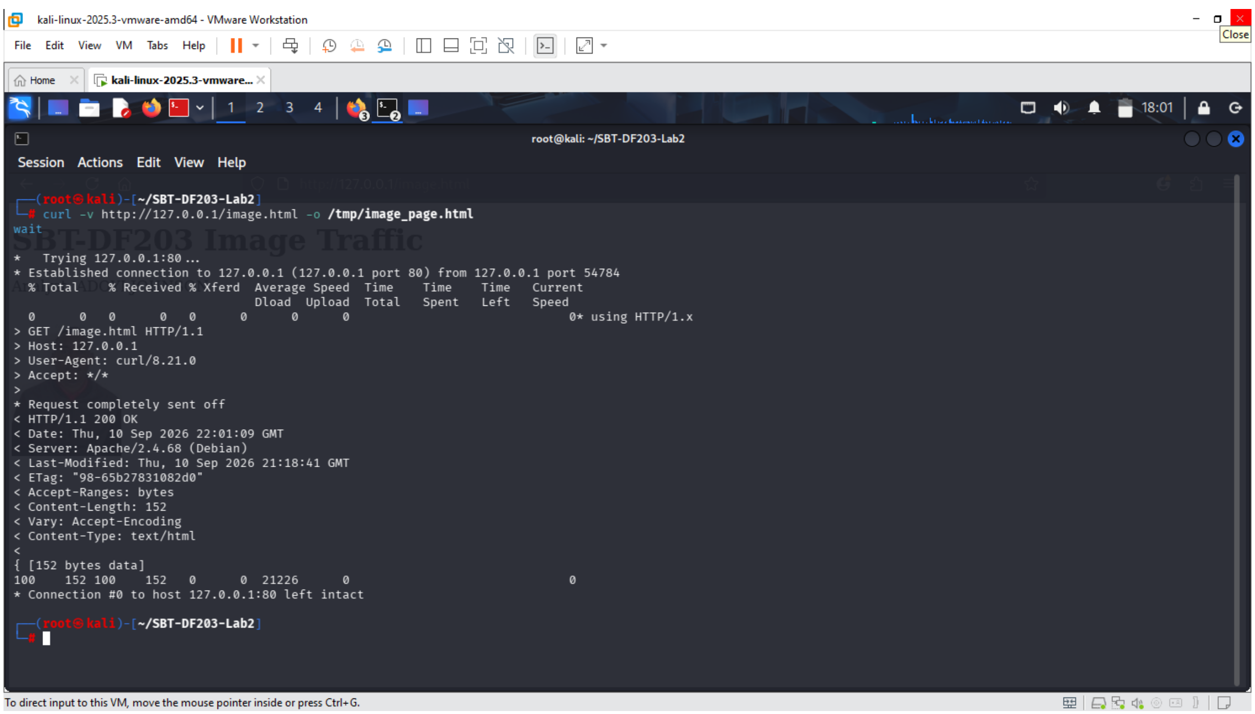
Run curl against image.html in a new capture. By default, curl retrieves the HTML but does not automatically fetch the referenced image. Explain why the image request is absent unless you request it separately.

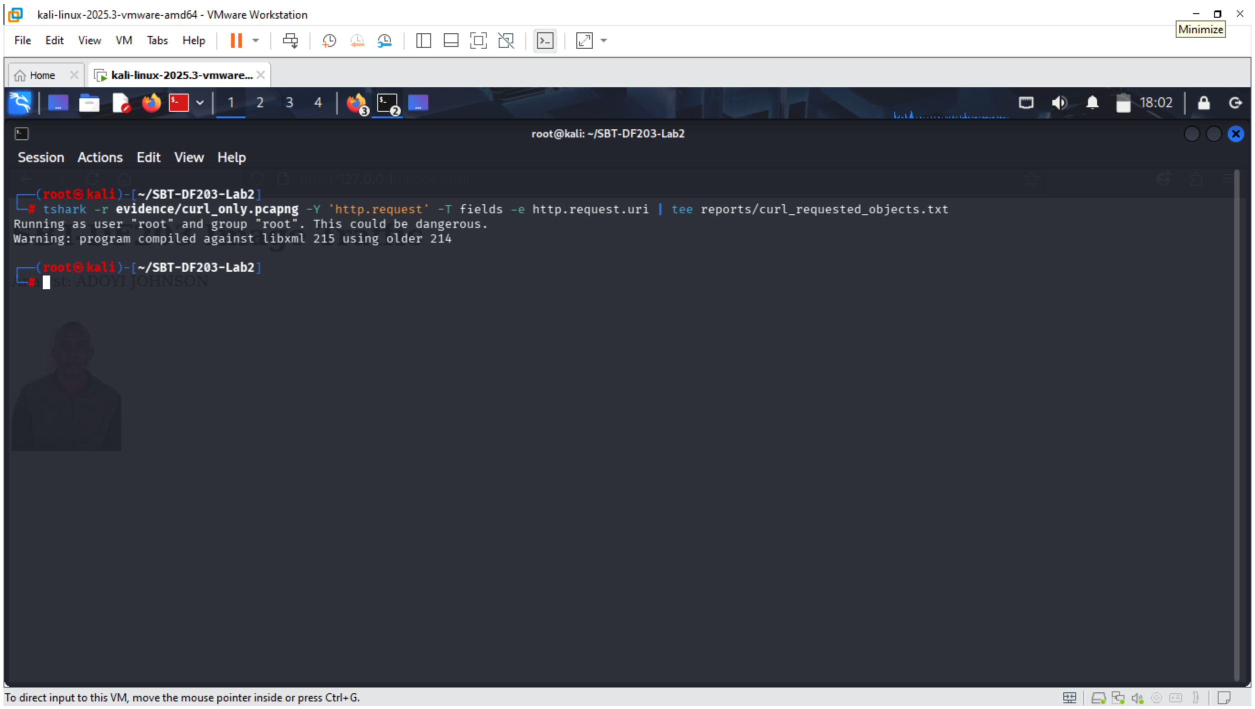
```
sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
sleep 2
curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
wait

tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e http.request.uri | tee
reports/curl_requested_objects.txt
```

ANSWER:


kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Home
kali-linux-2025.3-vmware...
17:59
root@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
┌──(root㉿kali)-[~/SBT-DF203-Lab2]
└─# sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
sleep 2
[1] 31026
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
┌──(root㉿kali)-[~/SBT-DF203-Lab2]
└─#
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.


kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Home
kali-linux-2025.3-vmware...
18:01
root@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
┌──(root㉿kali)-[~/SBT-DF203-Lab2]
└─# curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
wait
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 54784
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0* using HTTP/1.x
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 10 Sep 2026 22:01:09 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 10 Sep 2026 21:18:41 GMT
< ETag: "98-65b27831082d0"
< Accept-Ranges: bytes
< Content-Length: 152
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [152 bytes data]
100 152 100 152 0 0 21226 0 0
* Connection #0 to host 127.0.0.1:80 left intact
┌──(root㉿kali)-[~/SBT-DF203-Lab2]
└─#
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.



14. Required Forensic Findings

Finding	Student Observation
Number of HTTP requests	2
HTML request URI and timestamp	Date: Thu, 10 Sep 2026 22:04:21 GMT
Image request URI and timestamp	GET /image.html HTTP/1.1
HTTP response codes	[152 bytes data]
Image content type	Content-Type: text/html
Reported Content-Length	152
Number of reassembled TCP segments	4
Sum of TCP payload lengths	3
Original image SHA-256	7080ea1506205b21fd847fadf3757cedeae73c7452e12de45092f697d10daaa5 /var/www/html/image.html

Finding	Student Observation
Extracted image SHA-256	31f5a32c76f0077559a44ce280d7a90eacb24c553db96d43a880df69a01e786c /var/www/html/lab_johnson.jpg
Hash comparison conclusion	
curl/browser difference	

Required Screenshots Checklist

☐ Webpage showing the embedded image.
☐ Source image type, dimensions, size and hash.
☐ Capture of the browser request.
☐ HTTP request list showing HTML and image GETs.
☐ Image response headers.
☐ Reassembled TCP segment list.
☐ Export Objects window.
☐ Recovered image opened successfully.
☐ Original and recovered SHA-256 comparison.
☐ curl-only capture showing requested objects.

Student Report Template

13. Executive summary and scope.
14. Evidence acquisition and integrity hashes.
15. HTTP request/response inventory.
16. TCP segmentation and reassembly analysis.
17. Object extraction method.
18. Hash-based integrity conclusion.
19. curl versus browser explanation.
20. Limitations and recommendations.
21. Screenshots and command appendix.

Submission Requirements

- One PDF report named SBT-DF203-Lab2_RegNo_FullName.pdf.
- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.
- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
Preparation and source objects	10	Valid webpage and authorized image, with baseline hashes.
Capture quality	15	Complete browser capture with HTML and image requests.
HTTP analysis	20	Correct object requests, responses, types and timestamps.

Assessment Area	Marks	What the Instructor Will Check
Segmentation/reassembly	20	Accurate segment identification, lengths and explanation.
Object extraction and hashing	25	Recovered image opens and hash comparison is correctly interpreted.
Report quality	10	Professional evidence narrative and screenshots.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
Expected object requests	At least image.html and lab_photo.jpg.
Typical response status	HTTP 200 OK.
Image response type	image/jpeg or image/png.
Reassembly	One HTTP object may span multiple TCP segments.
Browser behaviour	Browser parses HTML and fetches embedded resources.
curl behaviour	curl normally retrieves only the explicitly requested URL.

Command Reference Summary

Command	Purpose
http.request	List requested HTTP objects.
http.response	List HTTP responses.
tcp.len	TCP payload length field.
File > Export Objects > HTTP	Recover transferred files.
tshark --export-objects http,DIR	Command-line HTTP object extraction.
sha256sum	Verify recovered object integrity.

Academic Integrity and Authorization Statement

Student Declaration
I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.